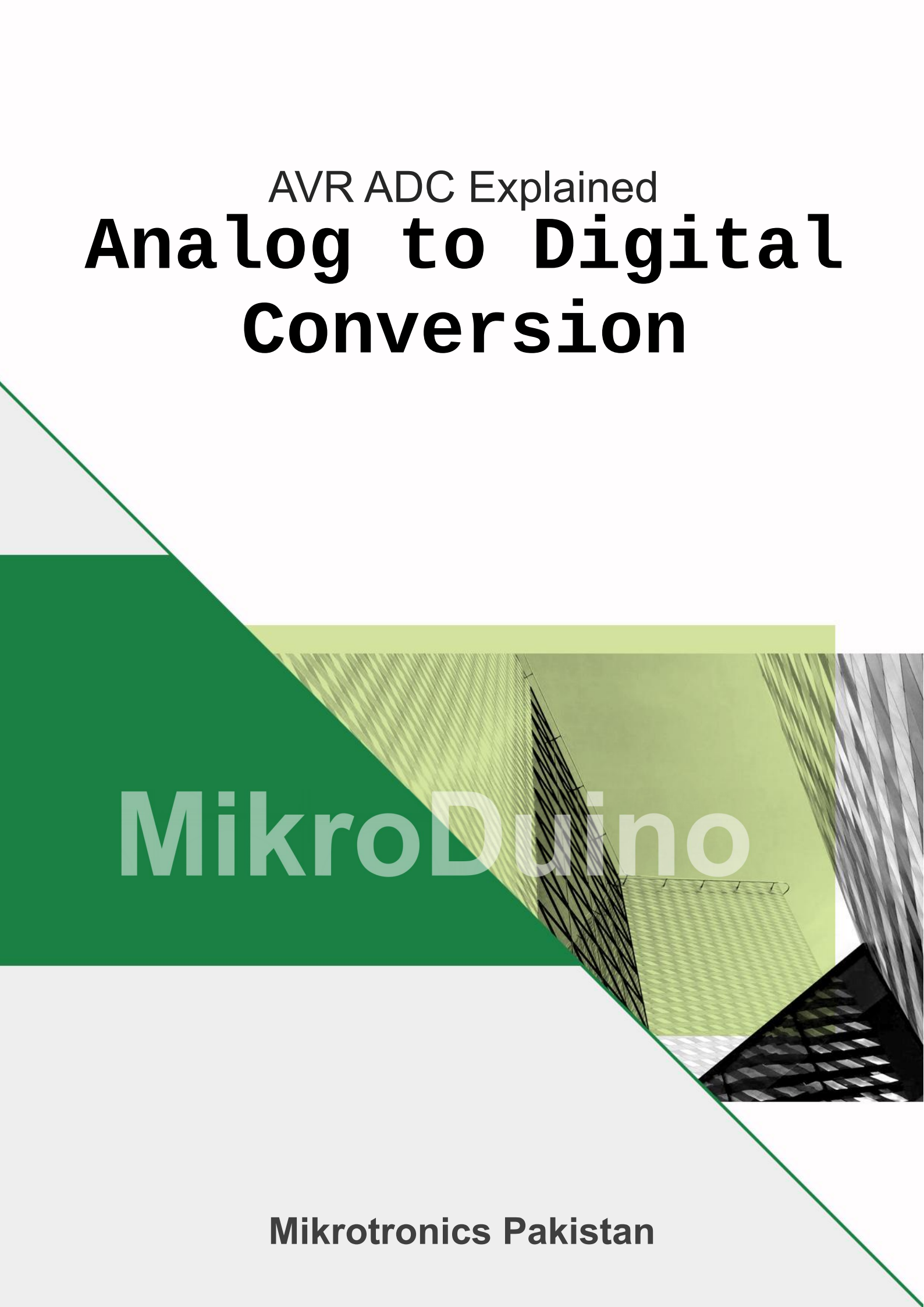


AVR ADC Explained

Analog to Digital Conversion



MikroDuino

Mikrotronics Pakistan

Analog to Digital Conversion | ADC

AVR Microcontrollers ADC Hardware

One of the greatest strengths of a microcontroller is its ability to interact with the real world. Unlike desktop computers that primarily process information entered through keyboards, mice, or network connections, embedded systems constantly observe and respond to their physical environment. They monitor temperature, detect light, measure pressure, sense motion, read battery voltage, listen to sound, and control countless devices that make modern life more convenient and efficient.

However, there is a fundamental challenge. The world around us is **analog**, while a microcontroller is **digital**.

An analog signal can take on any value within a continuous range. Consider the temperature in a room. It may be 22.1°C, 22.15°C, or 22.153°C. Similarly, the brightness of a lamp changes gradually, the voltage of a battery slowly decreases as it discharges, and the position of a joystick can lie anywhere between its minimum and maximum travel. Nature does not operate in discrete steps.

A microcontroller, on the other hand, understands only binary numbers. Every calculation, comparison, and decision it makes is based on digital values represented by combinations of zeros and ones. Before the microcontroller can process information from the outside world, the continuously varying analog signals must first be translated into digital numbers.

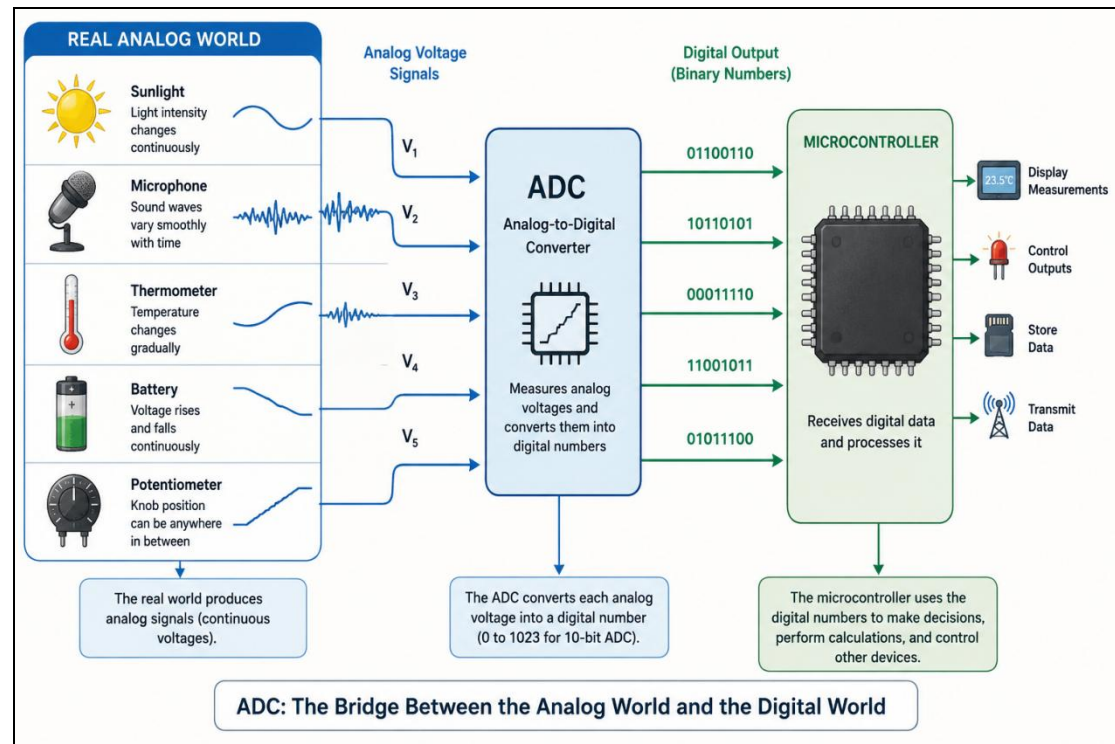
This translation is performed by a special hardware peripheral known as the **Analog-to-Digital Converter**, commonly abbreviated as the **ADC**.

The ADC measures an input voltage and converts it into a binary number that the CPU can process. From the software's point of view, an analog voltage becomes just another integer stored in memory. Once converted into digital form, the program can compare values, perform mathematical calculations, display measurements on a screen, store data for later analysis, or use the information to control actuators and other electronic devices.

It is important to understand that an ADC does **not** directly measure physical quantities such as temperature, pressure, humidity, or light intensity. Instead, it measures **voltage**. Sensors are responsible for converting physical phenomena into electrical signals. A temperature sensor generates a voltage proportional to temperature, a light sensor produces a voltage that varies with illumination, and a pressure sensor outputs a voltage corresponding to the applied pressure. The ADC simply converts these voltages into digital values, leaving it to the software to interpret their physical meaning.

The importance of ADCs in embedded systems cannot be overstated. Almost every intelligent electronic device relies on analog measurements in some form. A digital thermometer continuously converts sensor voltages into temperature readings. An automotive engine control unit monitors numerous analog sensors to optimize fuel

injection and ignition timing. Medical instruments acquire biological signals through ADCs before analyzing them. Smartphones measure battery voltage, ambient light, microphone input, and touch-screen signals using multiple analog-to-digital converters. Industrial automation systems constantly monitor voltages from pressure, flow, and position sensors to maintain safe and efficient operation.



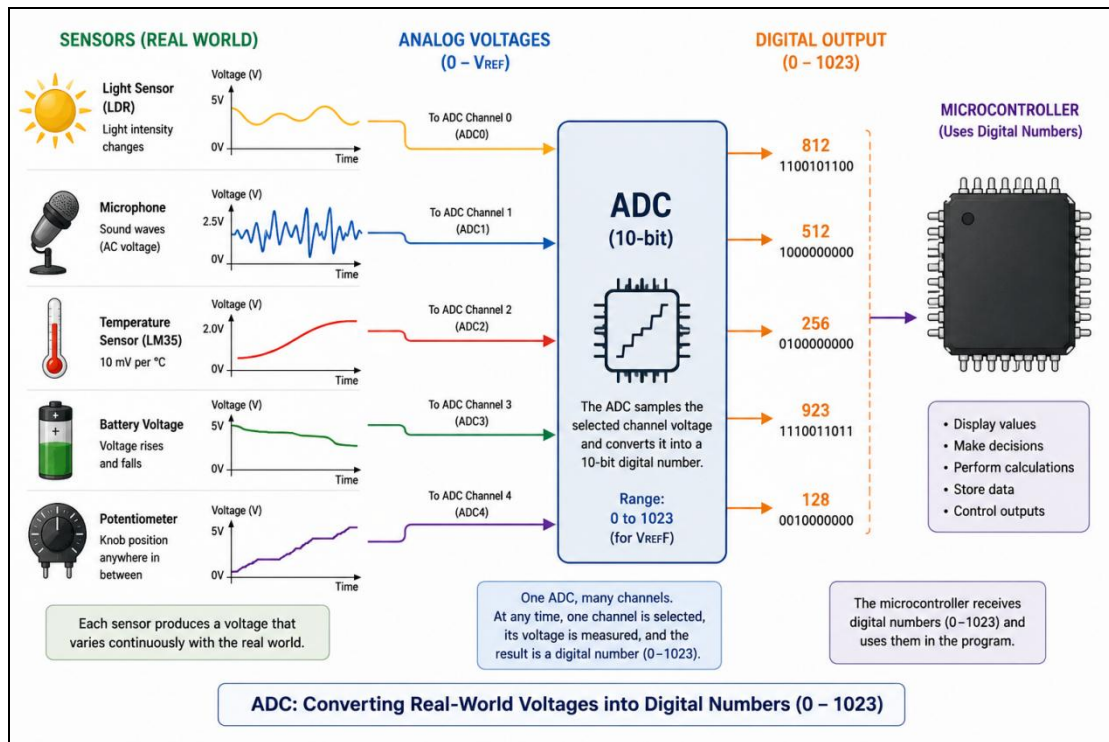
Without an ADC, a microcontroller would be largely isolated from the physical world. It could generate digital outputs to control LEDs, motors, or displays, but it would have no direct way of knowing the surrounding temperature, the brightness of a room, the position of a control knob, or the voltage of its own power supply. The ADC serves as the bridge that connects the digital intelligence of the microcontroller with the continuously changing analog world around it.

Throughout this chapter, we will explore how analog-to-digital conversion works in general, examine the powerful ADC hardware built into AVR microcontrollers, learn how to program it using standard AVR register-level programming, and finally discover how the **MikroDuino SDK** provides a modern, intuitive, and highly flexible interface that simplifies development while preserving complete control over the underlying hardware.

There are number of ways in which various sensors respond to the physical world. Some alter their conductivity, some alter their optical characteristics, some produce voltage or alter current, some tend to vibrate and change their frequency. In general the most common method of responding is voltage.

Sensor	Physical Quantity	Voltage Produced
LM35	Temperature	10 mV/° C
LDR	Light	Variable voltage

Sensor	Physical Quantity	Voltage Produced
Potentiometer	Position	Variable voltage
Microphone	Sound	Small AC voltage
Pressure Sensor	Pressure	Variable voltage



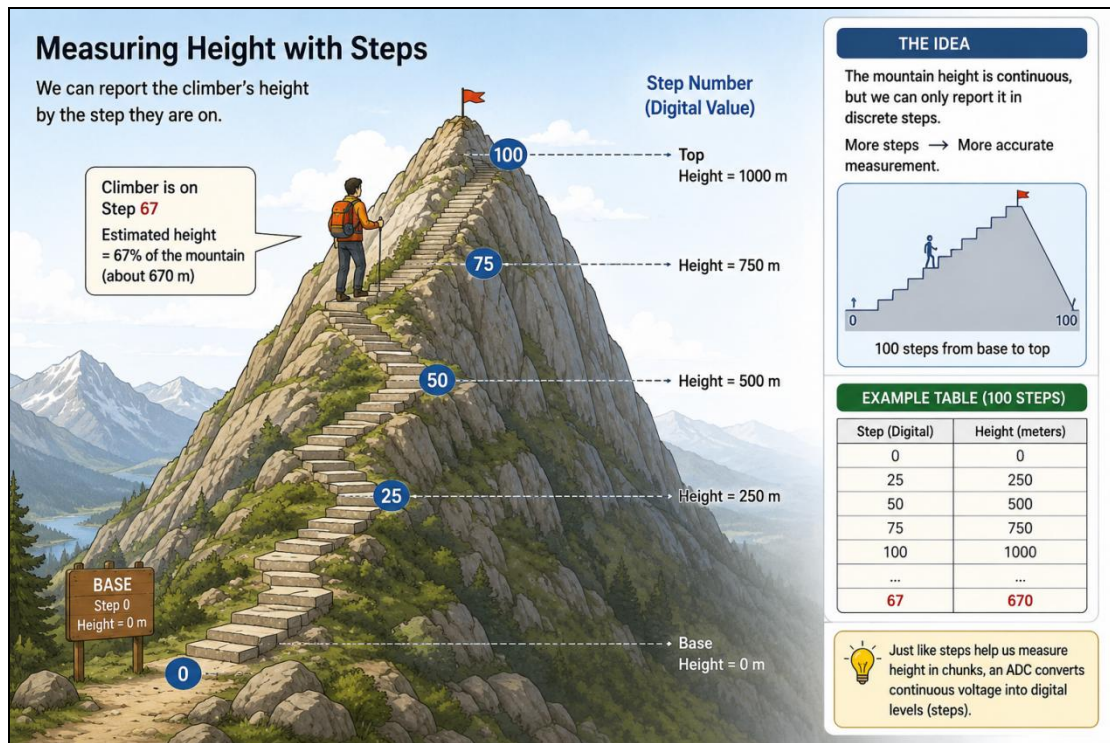
Digital Resolution — How Many Steps Can We See?

Imagine you are standing at the bottom of a mountain. Your friend is climbing upward, and every few seconds you are asked to report his height.

In one situation, you are only allowed to answer either **"At the bottom"** or **"At the top."** No matter where your friend actually stands, you must choose one of these two answers. Clearly, your description of his position is very poor.

Now imagine you are given ten steps to describe his height. Suddenly your answers become much more meaningful. You can say that he is on the third step, the seventh step, or the ninth step. Although you still cannot describe his exact position, your estimate has become much closer to reality.

Finally, imagine the mountain is divided into one thousand tiny steps. Now your estimate becomes remarkably accurate. Even though your friend can still stand between two steps, the difference is so small that your reported position is very close to the truth.



This simple story captures the essence of **digital resolution**.

An Analog-to-Digital Converter cannot represent every possible analog voltage. Instead, it divides the entire input voltage range into a fixed number of small intervals or **digital steps**. Each step is assigned a unique digital number.

The greater the number of steps, the finer the measurement becomes. This ability to distinguish between small changes in voltage is known as the **resolution** of the ADC.

Resolution Depends on the Number of Bits

The resolution of an ADC is determined entirely by the number of bits used to represent its output.

A converter with **N bits** can produce

$$[2^N]$$

different digital values.

For example,

ADC Resolution	Number of Digital Values
1-bit	2
2-bit	4
4-bit	16
8-bit	256

ADC Resolution	Number of Digital Values
10-bit	1,024
12-bit	4,096
16-bit	65,536

Notice how every additional bit doubles the number of available measurement levels. This is one of the reasons why increasing the ADC resolution greatly improves the ability to detect small changes in the input voltage.

Resolution of the AVR ADC

The AVR family of microcontrollers used by the MikroDuino platform contains a **10-bit Successive Approximation (SAR) ADC**. This means the converter divides the entire reference voltage into **1,024 discrete levels**, numbered from

0 1023

If the reference voltage is **5.0 volts**, then the ADC interprets the input voltage as follows:

0.000 V → 0
5.000 V → 1023

Every voltage between these limits is represented by one of the 1,024 digital numbers.

The mathematical relationship is

$$[ADC = \frac{V_{IN}}{V_{REF}} \times 1023]$$

where

V_{IN} is the input voltage.

V_{REF} is the selected reference voltage.

1023 is the largest value that can be represented by a 10-bit ADC.

An Example

Suppose the AVR uses **AVCC = 5 V** as its reference voltage.

If we apply exactly

2.5 V

to an analog input pin, the ADC calculates

$$[ADC = \frac{2.5}{5.0} \times 1023]$$

which gives approximately

512

Similarly,

Input Voltage	Digital Value
0.0 V	0
1.25 V	256
2.50 V	512
3.75 V	768
5.00 V	1023

Notice that the output is simply a number representing the position of the input voltage between **0 V** and the reference voltage.

The Smallest Detectable Change

Since the AVR divides the entire voltage range into 1,024 steps, each step represents only a small change in voltage.

This smallest measurable change is called the **resolution per count**.

It is calculated as

$$[\text{Resolution} = \frac{V_{\text{REF}}}{1024}]$$

For a 5-volt reference,

$$[\text{Resolution} = \frac{5.0}{1024}]$$

which equals approximately

4.88 mV

This means that every increase of one ADC count corresponds to about **4.88 millivolts**.

A voltage change smaller than this usually cannot be distinguished by the converter because both voltages will produce exactly the same digital value.

If the reference voltage is changed to **1.1 V**, the resolution becomes much finer:

$$[\frac{1.1}{1024} \approx 1.07 \text{mV}]$$

The converter itself has not changed—it is still a 10-bit ADC—but by reducing the measurement range, each digital step now represents a much smaller voltage difference.

This is one of the reasons why selecting an appropriate reference voltage is extremely important in precision measurements.

More Bits Mean More Detail

It is tempting to think that higher resolution always means greater accuracy. In reality, resolution and accuracy are not the same thing.

Resolution tells us **how finely the voltage range is divided**.

Accuracy tells us **how close the measured value is to the true voltage**.

An inaccurate ADC can have very high resolution, just as a ruler with very fine markings can still produce incorrect measurements if it is poorly manufactured or improperly calibrated.

Throughout this chapter, we will focus first on obtaining the full 10-bit resolution offered by the AVR ADC. Later, when we discuss reference voltages, sampling techniques, and noise reduction, we will see how to achieve the best possible accuracy from the hardware.

Sampling — Taking Snapshots of the Analog World

Imagine standing beside a busy highway with a camera in your hands. Your task is to observe a passing car and determine how fast it is moving. If you take only **one photograph**, you will know where the car was at that instant, but you will have no idea how it moved before or after the picture was taken.

Now suppose you take a photograph every **ten seconds**. The car will have travelled a long distance between photographs. From these widely spaced snapshots, it becomes difficult to reconstruct its journey. In fact, you might even conclude that the car suddenly jumped from one place to another.

Finally, imagine using a high-speed camera that captures **hundreds of photographs every second**. When these images are played back one after another, the motion appears perfectly smooth. Every tiny movement of the car has been recorded.

This is exactly how an Analog-to-Digital Converter observes the analog world.

Unlike our eyes, which continuously perceive changing light, an ADC cannot watch a voltage continuously. The hardware simply does not operate that way. Instead, it measures the input voltage at specific moments in time.

Each individual measurement is called a **sample**.

Between two samples, the ADC is effectively "blind." It does not know what happened to the voltage until it performs the next measurement.

You can think of an ADC as a very fast digital camera. Instead of taking photographs, it takes **voltage snapshots**. Every snapshot is converted into a digital number and stored or processed by the microcontroller.

If these snapshots are taken frequently enough, the software can accurately reconstruct how the voltage changes with time.

What Happens During Sampling?

Consider a temperature sensor connected to an AVR microcontroller.

The room temperature changes very slowly. Taking one sample every second is more than sufficient because the temperature hardly changes during that short interval.

Now consider a microphone connected to the same ADC.

Sound waves change thousands of times every second. If the ADC samples the microphone only once every second, it will completely miss the sound. Even sampling ten times per second would capture only a few isolated points that bear little resemblance to the original waveform.

Different signals therefore require different sampling rates.

Slowly changing signals such as temperature, humidity, battery voltage, or soil moisture can be sampled infrequently. Rapidly changing signals such as music, speech, vibration, or radio signals require much faster sampling.

The art of choosing an appropriate sampling rate is one of the most important aspects of embedded system design.

Sampling Too Slowly

Imagine watching a movie that displays only one frame every two seconds.

People would suddenly appear in different positions. Cars would seem to teleport down the road. The movement would no longer appear continuous.

Exactly the same thing happens when an ADC samples too slowly. The digital measurements no longer represent the actual analog signal. Important details are lost forever because the converter simply never observed them.

This loss of information cannot be recovered later. Once a signal has been sampled too slowly, no amount of software processing can reconstruct what was never measured.

Sampling Fast Enough

Now imagine watching a modern movie running at sixty frames every second. The individual images blend together so smoothly that our eyes perceive continuous motion.

An ADC behaves in much the same way.

If it samples the input voltage rapidly enough, the sequence of digital values closely follows the original analog waveform. The microcontroller can then process, display, or analyze the signal with confidence that it accurately represents the real world.

Fortunately, the ADC built into AVR microcontrollers is capable of taking thousands of samples every second, making it suitable for a wide variety of sensing applications.

The Nyquist Sampling Theorem

One of the most important ideas in digital signal processing was introduced by the engineer **Harry Nyquist**.

Fortunately, the basic idea is surprisingly simple.

To accurately reproduce a changing signal, the sampling rate must be at least twice the highest frequency present in that signal.

This principle is known as the **Nyquist Sampling Theorem**.

For example, suppose the highest frequency contained in a signal is **1,000 Hz**.

According to Nyquist, the ADC should sample at **at least 2,000 samples per second**. Sampling faster is perfectly acceptable—and often desirable—but sampling slower risks losing information and producing misleading results.

A familiar example is digital audio. Human hearing extends to approximately **20 kHz**, so audio systems typically sample at **44.1 kHz**, more than twice the highest audible frequency. This is why CDs are able to reproduce music with excellent fidelity.

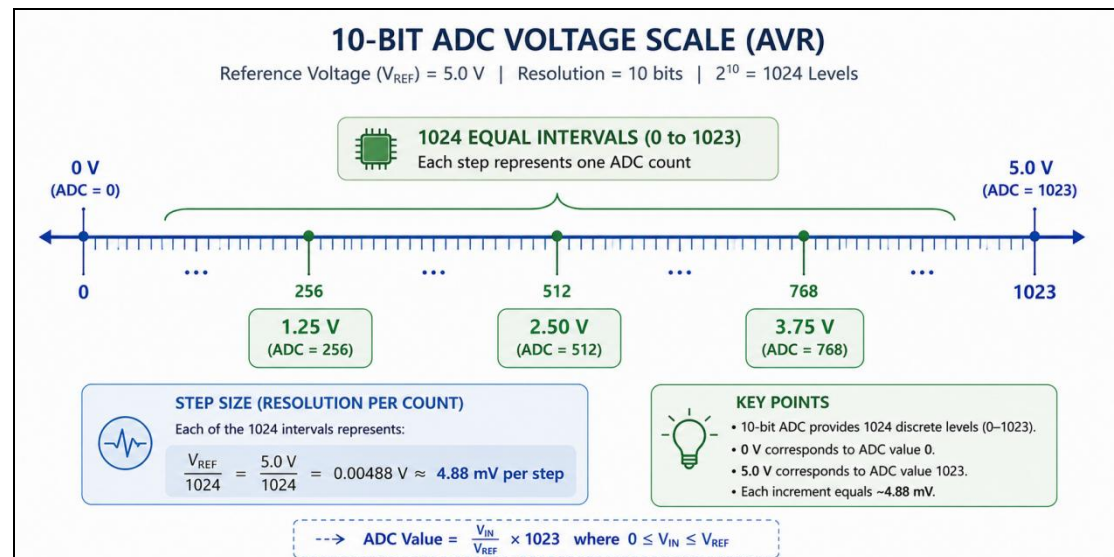
What Happens If We Ignore Nyquist?

Suppose a wheel is spinning rapidly while a camera takes only a few pictures each second. Sometimes the wheel appears to rotate slowly. Sometimes it seems to stop. Occasionally it even appears to rotate backwards.

Of course, the wheel has not changed its motion—the camera simply failed to capture enough snapshots. This fascinating illusion is called **aliasing**, and exactly the same phenomenon occurs in ADC systems.

When the sampling rate is too low, the measured digital signal no longer represents the original analog signal. Instead, it may appear to have an entirely different frequency or shape.

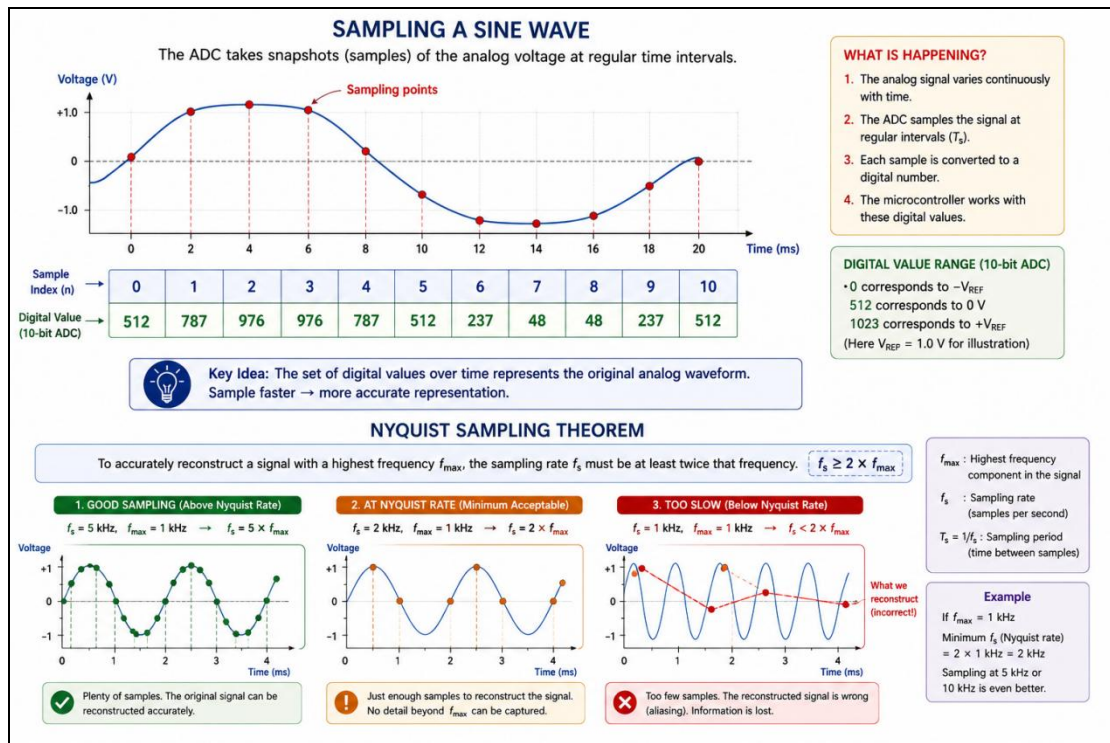
For this reason, engineers almost always choose a sampling rate comfortably higher than the theoretical Nyquist limit whenever possible.



Sampling in AVR Microcontrollers

The AVR's 10-bit Successive Approximation ADC performs one conversion in approximately **13 ADC clock cycles** after the first conversion. Depending on the selected ADC clock frequency, this allows the microcontroller to acquire thousands of samples every second.

For many embedded applications—such as measuring temperature, light intensity, battery voltage, potentiometers, or pressure sensors—this sampling speed is more than adequate. Even for more demanding applications such as audio processing or waveform analysis, careful selection of the ADC clock and sampling interval enables the AVR to capture surprisingly detailed information.



The Successive Approximation ADC — A Smart Way of Guessing

Suppose I ask you to guess a number that I am thinking of. The number lies somewhere between **1** and **1024**.

If your strategy is to start at **1** and keep counting upward—

"Is it 1?"

"No."

"Is it 2?"

"No."

"Is it 3?"

"No."

—you might have to ask hundreds of questions before arriving at the correct answer. In the worst case, you may need to ask all 1,024 questions. Clearly, this is not a very intelligent approach.

Now imagine a different strategy.

Instead of starting at one end, you immediately ask,

"Is your number greater than 512?"

If the answer is **Yes**, you instantly eliminate half of all possible numbers.

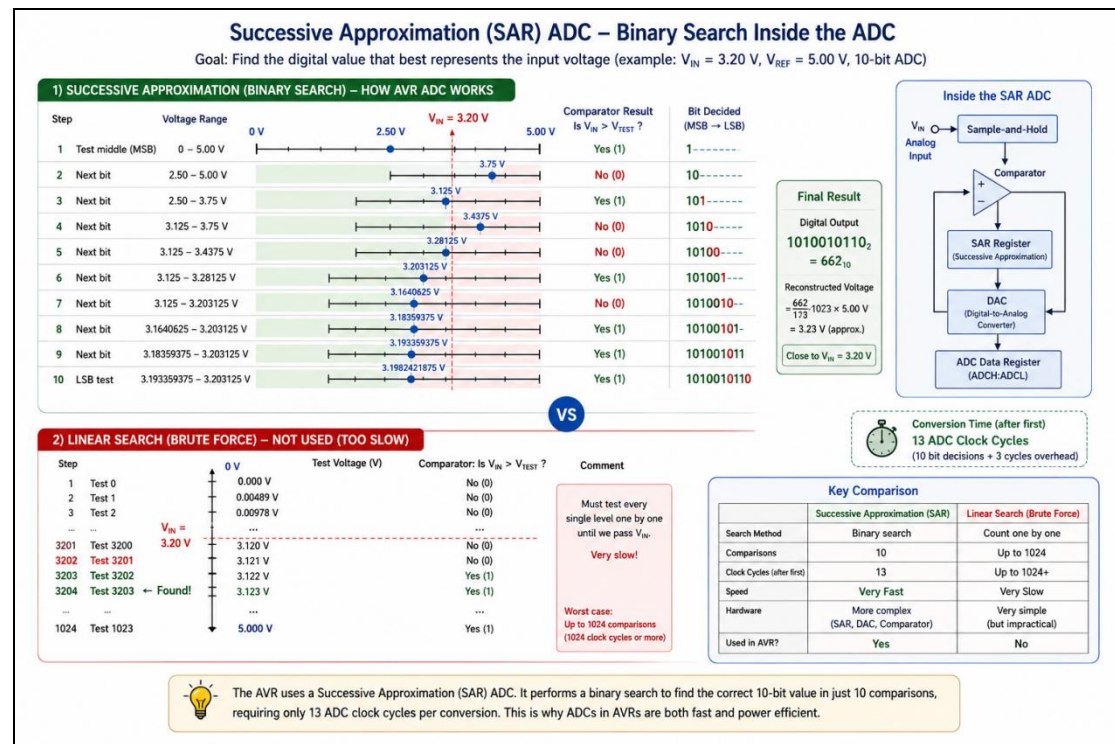
Now you know the answer lies somewhere between **513** and **1024**.

Next you ask,

"Is it greater than 768?"

Again, half of the remaining possibilities disappear.

You continue dividing the search range into smaller and smaller halves.



Within only a handful of questions, you have discovered the exact number.

Computer scientists call this technique **Binary Search**, and it is one of the fastest searching algorithms ever invented. Surprisingly, the ADC inside an AVR microcontroller uses almost exactly the same strategy. Instead of guessing a number, it guesses an **analog voltage**.

Searching for a Voltage

The analog voltage connected to an ADC input can be any value between **0 volts** and the selected **reference voltage**. For example, if the reference voltage is **5 volts**, the input could be

- ✓ 0.12 V
- ✓ 1.83 V
- ✓ 2.47 V
- ✓ 3.96 V

or any other value within that range.

The ADC's task is to determine which one of the **1,024 possible digital levels** best represents that voltage.

Rather than comparing the input voltage against every possible level one after another, the AVR uses a much smarter approach. It repeatedly divides the remaining search range into two equal halves until only one answer remains.

This method is called the **Successive Approximation Register**, or simply the **SAR ADC**.

Why Is It Called "Successive Approximation"?

The word **approximation** means making the closest possible estimate. The word **successive** means improving that estimate one step after another. The ADC therefore begins with a rough estimate of the input voltage and then repeatedly improves that estimate until it reaches the final answer.

Each new comparison reduces the remaining uncertainty by half. After enough comparisons, only one digital value remains.

How the AVR ADC Thinks

Let us imagine that the input voltage is actually **3.2 volts** and the reference voltage is

5.0 volts. The ADC does not know this value beforehand. It begins its search.

Step 1 — Guess the Middle

The first question is simple.

"Is the input voltage greater than half of the reference voltage?"

Half of 5 volts is **2.5 V**.

Since

3.2 V > 2.5 V

the answer is: **Yes.**

The ADC immediately knows that the voltage lies in the **upper half** of the measurement range.

The most significant bit (MSB) is therefore set to **1**.

Step 2 — Search the Upper Half

Now the remaining search range is

2.5 V ----- 5 V

The middle of this new range is

3.75 V.

The ADC asks,

"Is the input voltage greater than 3.75 V?"

This time,

3.2 V < 3.75 V

The answer is: **No.**

The ADC now knows the voltage lies between 2.5 V and 3.75 V.

The second bit becomes **0**.

Step 3 — Divide Again

The remaining range is divided once more.

2.5 -----3.75

Middle

3.125 V

Now,

3.2 V > 3.125 V

The answer is: **Yes.**

The next bit becomes **1**.

Again, half of the remaining search space disappears.

Continuing the Search

The process repeats over and over. Each comparison decides exactly **one bit** of the final digital number. Every new decision halves the remaining uncertainty.

After ten such decisions, all ten output bits have been determined. The ADC has found the closest digital representation of the input voltage.

The Successive Approximation Register (SAR)

Where are these bits stored while the ADC is making its decisions?

Inside a special piece of hardware called the **Successive Approximation Register**, or **SAR**. The SAR is a small digital register that gradually builds the final 10-bit answer.

Initially, every bit is zero.

```
0000000000
```

After the first comparison,

```
1000000000
```

The next comparison determines whether the second bit should remain set or be cleared.

```
1100000000
```

or

```
1000000000
```

One bit at a time, the register fills with the correct binary pattern. By the end of the conversion, it may contain something like

```
1010001110
```

This binary value is then copied into the ADC Data Registers (**ADCL** and **ADCH**) where the CPU can read it.

Why Is SAR So Popular?

There are many different types of Analog-to-Digital Converters.

Some are extremely fast. Others are incredibly accurate. Some consume very little power.

The Successive Approximation ADC offers an excellent balance between all of these characteristics. For this reason, SAR ADCs are found in countless embedded systems, including the AVR family of microcontrollers.

Their advantages include:

- ✓ Good conversion speed.
- ✓ Excellent power efficiency.
- ✓ Simple hardware design.
- ✓ High accuracy for embedded applications.
- ✓ Suitable for measuring slowly and moderately changing analog signals.

This makes the SAR ADC an ideal choice for applications such as temperature measurement, light sensing, battery monitoring, joystick reading, pressure sensors, industrial instrumentation, and hundreds of other embedded projects.

How Long Does One Conversion Take?

One of the most impressive aspects of the AVR ADC is its speed. After the ADC has been enabled, each conversion requires approximately **13 ADC clock cycles**. The first conversion after enabling the ADC takes a little longer because some internal circuitry must be initialized. Once the converter is running, however, every subsequent conversion completes in only thirteen ADC clock cycles.

Suppose the ADC clock is configured to **125 kHz**, which is a commonly recommended operating frequency for maximum accuracy. One ADC clock period is

$$\left[\frac{1}{125000}\right]=8\ \mu\text{s}$$

Since one conversion requires 13 clock cycles,

$$[13\times 8\ \mu\text{s}=104\ \mu\text{s}]$$

This means the AVR can perform approximately

9,600 analog conversions every second.

This speed is more than sufficient for most embedded applications, including sensor measurements, battery monitoring, data logging, motor control, and even basic audio acquisition.

The Intelligence Inside Every Conversion

Although reading an analog input in software may appear as simple as calling a single function, an astonishing amount of work is happening inside the microcontroller.

Within a few microseconds, the ADC repeatedly compares voltages, performs a binary search, updates the Successive Approximation Register one bit at a time, and finally delivers a precise 10-bit digital value to the CPU.

All of this happens automatically, allowing your program to receive an accurate measurement with almost no effort. The AVR ADC Hardware

The AVR ADC Hardware

Inside the AVR ADC — How the Microcontroller Measures Voltage

Up to this point, we have looked at the Analog-to-Digital Converter from the outside. We have learned that it measures an analog voltage, periodically samples it, performs a binary search using the Successive Approximation algorithm, and finally produces a 10-bit digital number.

From the programmer's perspective, this appears wonderfully simple. A single function call—or a few lines of register configuration—is enough to obtain a measurement.

But behind this simplicity lies a remarkably sophisticated piece of hardware.

When you ask the AVR microcontroller to perform an ADC conversion, dozens of tiny electronic circuits immediately begin working together. Some select which input pin should be measured. Others connect the chosen voltage to the converter. Another circuit stores the voltage momentarily while the conversion is taking place. A precision comparator repeatedly compares voltages. The Successive Approximation Register builds the digital result one bit at a time. Finally, the completed result is placed into memory where your program can read it.

All of these operations happen automatically in just a few microseconds.

Understanding this internal journey not only helps you write better software, but also explains why certain registers must be configured correctly, why the ADC requires a suitable clock frequency, why a reference voltage is necessary, and why some conversions are more accurate than others.

Let us now take a journey inside the AVR ADC and follow a single voltage measurement from beginning to end.

The Journey of an Analog Voltage

Imagine a small potentiometer connected to the **ADC0** input of an ATmega328P microcontroller. As you rotate the knob, the voltage on the pin changes smoothly between **0 volts** and **5 volts**.

Suppose the voltage happens to be **3.2 volts**.

When your program starts an ADC conversion, the following sequence begins.

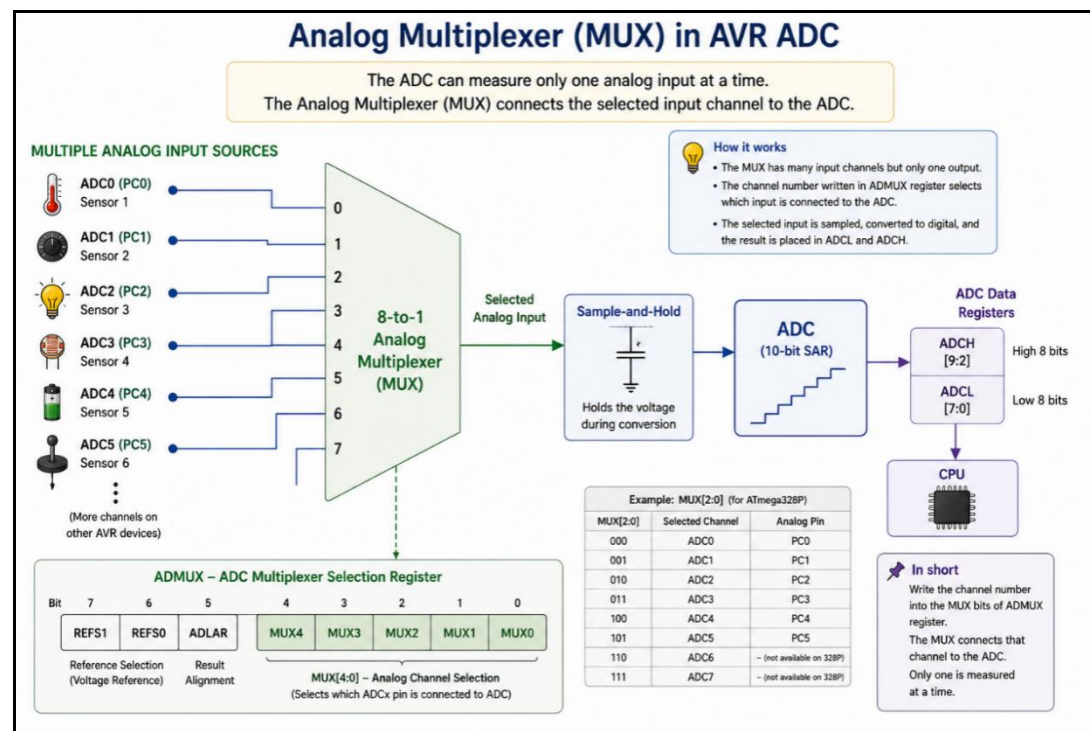
Step 1 — Selecting the Input Channel

The AVR contains multiple analog input pins. Depending on the microcontroller model, there may be six, eight, or even more analog channels. Obviously, the ADC can measure only **one voltage at a time**. So the first task is deciding **which pin** should be connected to the converter.

This job is performed by an electronic switch called the **Analog Multiplexer**, often abbreviated as the **MUX**. You can think of the multiplexer as a railway junction. Many railway tracks enter the station, but only one track can be connected to the platform at a time.

Likewise, many analog input pins are available, but the multiplexer connects only one of them to the ADC. Your software selects the desired channel by programming the **MUX bits** inside the **ADMUX register**.

Once selected, the chosen analog voltage begins its journey into the converter.



Step 2 — Choosing the Measuring Scale

Before measuring any voltage, the ADC must answer an important question.

"Compared to what?"

Measuring **3 volts** has little meaning unless we know the full measurement range.

For example,

- 3 volts out of 5 volts is quite different from

- 3 volts out of 10 volts.

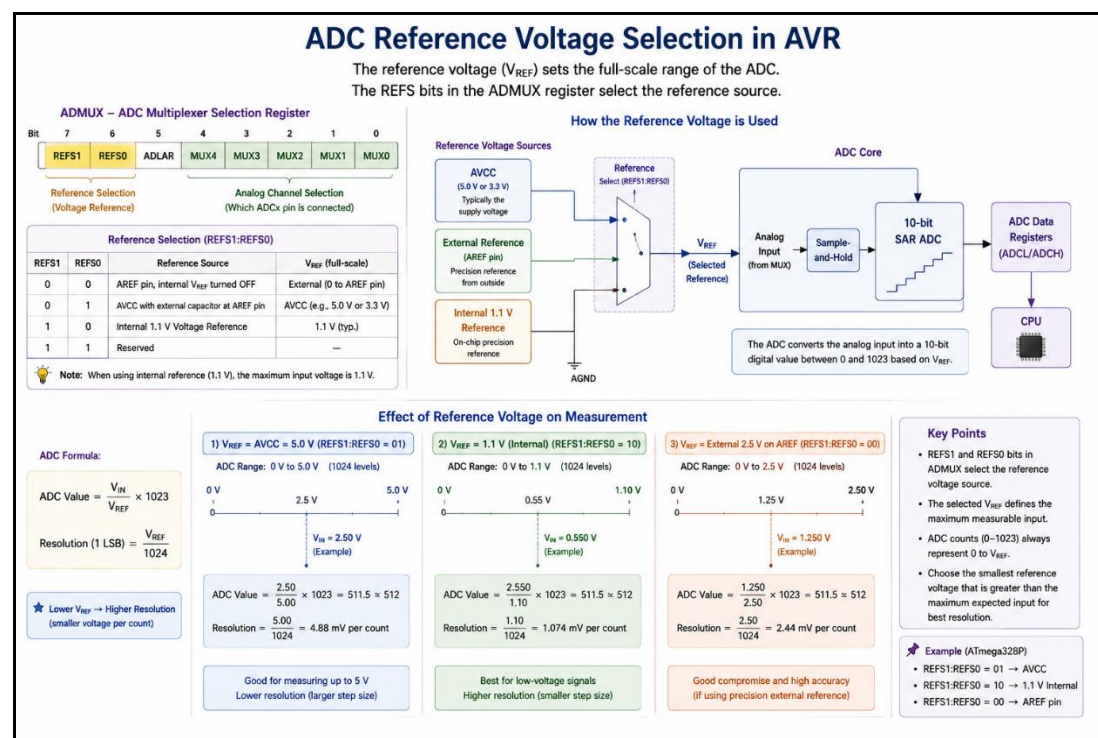
This upper limit is called the **Reference Voltage (VREF)**. Every ADC measurement is expressed as a fraction of this reference voltage. The AVR allows the programmer to choose between several reference sources.

For example,

- AVCC (usually 5 V or 3.3 V)
- An external precision reference connected to the AREF pin
- An internal precision voltage reference (typically 1.1 V)

The selected reference determines both the measurement range and the resolution. Choosing a smaller reference voltage allows much finer measurements of low-voltage sensors.

The reference source is selected using the **REFS bits** located in the **ADMUX register**.



Step 3 — Capturing the Voltage

One interesting question remains. What happens if the input voltage changes while the ADC is still converting? Suppose the potentiometer is being rotated continuously.

The ADC cannot allow the voltage to change halfway through its binary search because the final answer would be meaningless. To solve this problem, the AVR contains a tiny circuit called the **Sample-and-Hold** stage.

The instant a conversion begins; this circuit briefly stores the input voltage on a small internal capacitor.

From that moment onward, the ADC no longer measures the changing input pin. Instead, it measures the stable voltage stored on this capacitor. Only after the conversion is complete is the capacitor released, ready to capture the next sample.

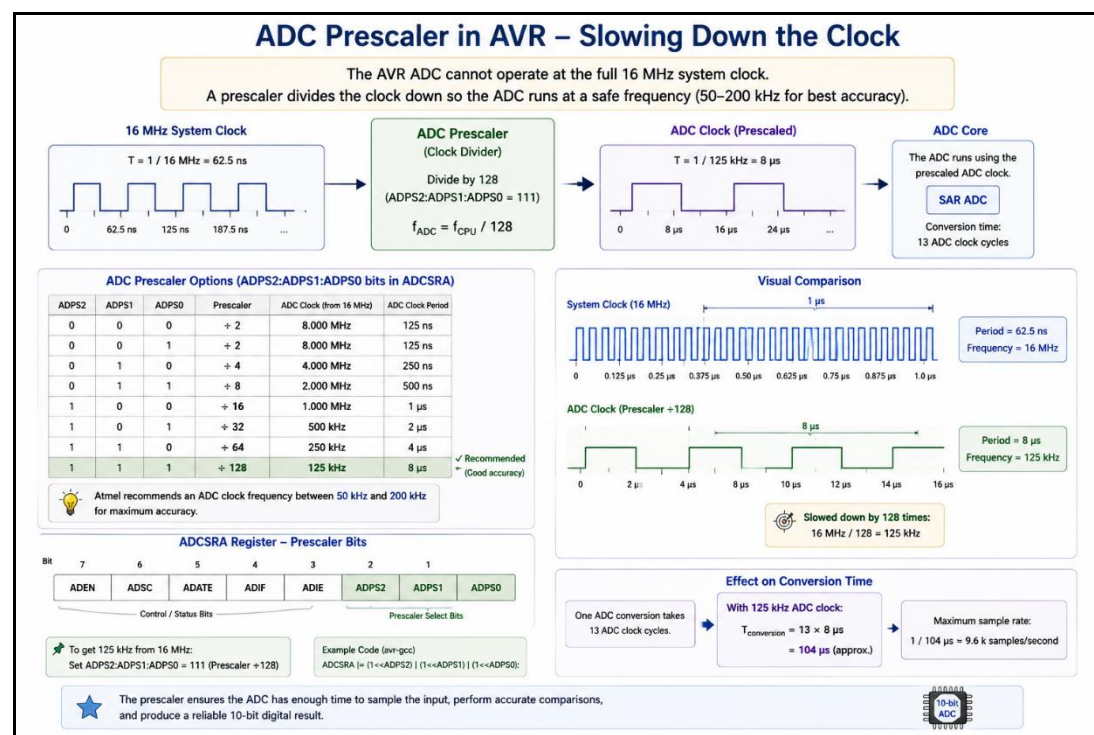
This is why every ADC conversion represents a precise snapshot of the analog signal.

Step 4 — Slowing Things Down

At first glance, one might think the ADC should operate as fast as possible. Surprisingly, this is not true. The CPU inside an AVR may be running at **16 MHz**, which means it executes sixteen million clock cycles every second.

The analog circuitry inside the ADC cannot operate accurately at such high frequencies. Instead, it requires a much slower clock.

The solution is a **Prescaler**. The prescaler divides the main CPU clock before sending it to the ADC.



This slower clock gives the comparator and internal circuitry enough time to settle accurately during each comparison.

The prescaler value is selected using the **ADPS bits** inside the **ADCSRA register**.

For maximum accuracy, Atmel recommends an ADC clock between approximately **50 kHz and 200 kHz**.

Step 5 — The Successive Approximation Engine

Now the real work begins. The stored voltage enters the comparator. The Successive Approximation Register performs its binary search exactly as we discussed in the previous section. One bit is determined after another.

Every comparison removes half of the remaining uncertainty. Within only ten comparisons, the converter has identified the closest 10-bit digital value corresponding to the sampled voltage.

This remarkable process happens automatically in hardware. No software intervention is required.

Step 6 — Storing the Result

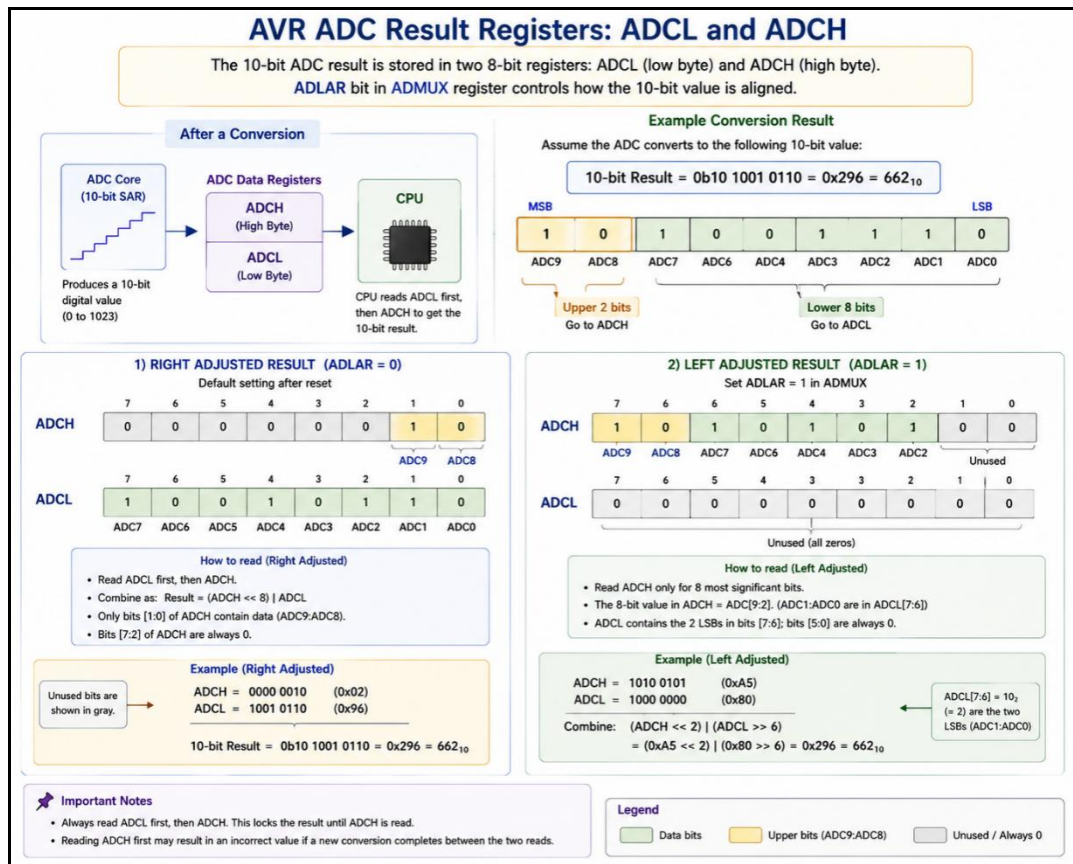
Once the conversion finishes, the resulting digital value is copied into two special registers.

These are

- **ADCL** (ADC Data Register Low)
- **ADCH** (ADC Data Register High)

Together they contain the complete 10-bit conversion result. Your program simply reads these registers to obtain the measured value.

The AVR even combines these two registers into a convenient 16-bit register named **ADC**, allowing the compiler to read both bytes as a single variable.



Step 7 — Informing the CPU

The ADC has now finished its work. How does the CPU know? The AVR provides two methods.

The simplest method is called **polling**. The program repeatedly checks a status bit until the conversion is complete.

The second method uses an **interrupt**. Instead of waiting, the CPU continues performing other tasks.

When the conversion finishes, the ADC automatically interrupts the processor and announces,

"The measurement is ready."

Interrupt-driven ADC programming becomes especially useful in high-speed data acquisition and real-time embedded systems.

The AVR ADC Hardware at a Glance

Although the ADC contains many internal circuits, its operation can be understood by following the flow of information.



Every conversion follows this same journey.

The only difference is which input channel is selected, which reference voltage is used, and how fast the ADC clock is running.

The Registers Behind the Scenes

While all of these operations occur automatically, the programmer controls the ADC through a small collection of registers.

Fortunately, each register has a clearly defined responsibility.

Register	Purpose
ADMUX	Selects the input channel, voltage reference, and data alignment.
ADCSRA	Enables the ADC, starts conversions, selects the prescaler, controls interrupts, and reports conversion status.
ADCSRB	Provides advanced features such as auto-trigger source selection and extended channel control on some AVR devices.
ADCL	Contains the lower eight bits of the conversion result.
ADCH	Contains the upper bits of the conversion result.
DIDR0	Disables the digital input buffers on analog pins to reduce power consumption and electrical noise during analog measurements.

Do not worry if these register names seem unfamiliar.

In the next sections, we shall examine each one individually, understand every bit it contains, and learn exactly how they work together to control the ADC.

Programming the AVR ADC Without the MikroDuino SDK

Talking Directly to the Hardware

So far in this chapter, we have explored what happens inside the microcontroller whenever an analog voltage is converted into a digital number.

We learned how the Analog Multiplexer selects one of several analog input pins. We discovered why the ADC requires a reference voltage, how the prescaler slows down the clock for accurate operation, and how the Successive Approximation Register performs a remarkably efficient binary search to determine the digital value. Finally, we saw how the completed result is stored inside the **ADCL** and **ADCH** registers for the CPU to read.

At this point, you may be wondering,

"If the hardware already knows how to perform all these operations, how do we tell it what to do?"

The answer lies in the ADC registers.

The AVR microcontroller contains a collection of memory-mapped registers that act as the control panel for the ADC hardware. Writing different values into these registers changes the behavior of the converter. One bit enables the ADC. Another starts a conversion. Several bits select the analog input channel. Others choose the voltage reference, the clock prescaler, interrupt behavior, and many other operating modes.

In other words, programming the ADC is nothing more than configuring these registers correctly.

Why Learn Register Programming?

You might ask another important question.

"If the MikroDuino SDK provides easy-to-use ADC functions, why should I learn all these registers?"

This is a perfectly reasonable question. The answer is simple. A good engineer should understand the hardware before relying on a software library.

Imagine learning to drive a car that has an automatic transmission. You can certainly drive from one place to another without understanding how gears, clutches, and differentials work. However, if something goes wrong, or if you wish to improve performance, that deeper understanding becomes invaluable.

Programming the ADC registers gives you exactly that understanding. It teaches you **how the hardware thinks**. Later, when we begin using the MikroDuino SDK, you will immediately recognize that every high-level function is simply performing the same register operations automatically.

Rather than hiding the hardware from you, the SDK builds upon your understanding of it.

The Registers Are the Language of the Hardware

Every peripheral inside an AVR microcontroller has its own collection of registers. The ADC is no exception. Think of these registers as the control panel of a sophisticated scientific instrument. Every switch has a purpose. Every button controls a particular feature. Every indicator reports the current status.

When we write

```
ADMUX = (1 << REFS0);
```

we are not calling a function.

We are physically changing the electrical configuration of the ADC.

The hardware immediately notices that the **REFS0** bit has become one and connects **AVCC** as the reference voltage.

Similarly,

```
ADCSRA |= (1 << ADSC);
```

does not execute an algorithm. Instead, it tells the ADC hardware,

"Begin a new conversion now."

The converter immediately starts sampling the selected analog input. This is one of the fascinating aspects of embedded programming. Writing software directly changes the behavior of electronic circuits.

The Six Steps of Every ADC Conversion

Although the ADC contains many advanced features, every single conversion follows exactly the same basic sequence. Regardless of whether you are measuring a potentiometer, a temperature sensor, or a battery voltage, the software performs these six steps.

Step 1 — Configure the Voltage Reference

The ADC must first know the maximum voltage it should measure. This is done by selecting the appropriate reference voltage using the **REFS** bits in the **ADMUX** register.

For most Arduino-compatible systems, AVCC is selected.

```
ADMUX = (1 << REFS0);
```

Step 2 — Select the Analog Channel

Next, the Analog Multiplexer must connect the desired analog input pin to the converter.

For example,

```
ADMUX |= 0; // ADC0
```

or

```
ADMUX |= 3; // ADC3
```

Only one channel can be measured at any given time.

Step 3 — Configure the ADC Clock

The ADC cannot operate directly from the 16 MHz CPU clock. Instead, we configure the prescaler.

For maximum accuracy,

```
ADCSRA =  
(1 << ADPS2) |  
(1 << ADPS1) |  
(1 << ADPS0);
```

This selects a divide-by-128 prescaler.

The ADC now operates at

```
16 MHz ÷ 128 = 125 kHz
```

which lies comfortably within Atmel's recommended operating range.

Step 4 — Enable the ADC

The converter remains switched off after reset. Before it can perform any measurement, it must first be enabled.

```
ADCSRA |= (1 << ADEN);
```

The ADC hardware is now ready.

Step 5 — Start a Conversion

Beginning a conversion is surprisingly simple.

```
ADCSRA |= (1 << ADSC);
```

The hardware immediately begins its work. The sample-and-hold circuit captures the input voltage. The Successive Approximation Register performs its binary search. The comparator repeatedly compares voltages.

Within only thirteen ADC clock cycles, the conversion is complete.

Step 6 — Read the Result

Once the conversion finishes, the result becomes available inside

ADCL and **ADCH**. Fortunately, AVR-GCC provides a convenient 16-bit register called

```
ADC
```

which automatically combines the two registers.

Reading the result therefore becomes wonderfully simple.

```
uint16_t value = ADC;
```

Your program now possesses a digital representation of the analog voltage.

Putting Everything Together

The complete sequence requires only a few lines of code.

```
ADMUX = (1 << REFS0);

ADCSRA =
    (1 << ADEN) |
    (1 << ADPS2) |
    (1 << ADPS1) |
    (1 << ADPS0);

ADCSRA |= (1 << ADSC);

while (ADCSRA & (1 << ADSC));
```

```
uint16_t value = ADC;
```

Although this program appears short, an extraordinary amount of work is taking place inside the microcontroller.

The ADC hardware has selected the correct input channel, connected the chosen reference voltage, slowed the clock using the prescaler, sampled the input voltage, performed ten successive comparisons using binary search, assembled the final 10-bit result, stored it in two hardware registers, and finally delivered the measurement to your C++ program.

All of this happened because we configured just a handful of registers.

Why Bare-Metal Programming Can Become Difficult

For a single conversion, register programming is straightforward. However, real applications are rarely this simple.

Suppose your project needs to:

- Measure six different sensors.
- Change the reference voltage.
- Switch between 8-bit and 10-bit modes.
- Perform continuous sampling.
- Use interrupts instead of polling.
- Automatically trigger conversions from a timer.
- Reduce power consumption.
- Support multiple AVR microcontrollers.

Suddenly, the program becomes filled with register manipulations. Bits must be set. Bits must be cleared. Registers must be updated in exactly the correct order. A single mistake can prevent the ADC from functioning correctly. As projects grow larger, the application logic becomes hidden beneath pages of low-level register code.

This is precisely why software libraries were invented. The goal is not to hide the hardware. The goal is to eliminate repetitive programming while still allowing the programmer complete control whenever it is needed.

In the next section, we shall discover how the **MikroDuino SDK ADC Library** accomplishes exactly that. It provides the simplicity of a modern C++ interface while remaining completely faithful to the underlying AVR hardware that you now thoroughly understand.